Project / Team Name

Project: DPDK Automation for Network Packet Processing **Organization:** Infobell IT Labs (engagement for AMD – CPU performance benchmarking) **Domain:** High-performance network packet processing & benchmark automation

Role: Lead Developer – Benchmark Automation **Dates:** April 2023 – September 2023

---

1. Business Opportunity (AMD DPDK Benchmark Automation)

- **Business goal – showcase AMD CPU networking performance**
  - AMD needed to **demonstrate and validate CPU performance** on **realistic, high-performance networking workloads** using DPDK and related benchmarks.
  - The objective was to have a **repeatable, automated framework** that can run multiple workloads (DPDK crypto, core library tests, testpmd, L2/L3 forwarding, iperf2/iperf3) and **compare results across CPU SKUs, BIOS settings, OSes, and compiler toolchains**.
- **Problem 1 – Manual runs were slow, fragile, and hard to reproduce**
  - Benchmark runs were **triggered manually**, which was **time-consuming and error-prone**, especially when testing many scenarios.
  - Changing BIOS configurations required **frequent reboots**, slowing down experimentation.
  - Multiple data points (benchmark output, CPU/power stats, system metrics) had to be captured in parallel, and **manual collection made back-tracking and reproducing results very difficult**.
- **Problem 2 – Intel-centric documentation and lack of AMD guidance**
  - Most existing DPDK benchmark documentation and tuning guides were **optimized for Intel platforms**.
  - There was **no consolidated, AMD-centric guide** on how to configure, tune, and run DPDK benchmarks on AMD CPUs, including AMD-specific BIOS/CPU features.
- **Problem 3 – Limited and incomplete existing automation frameworks**
  - Available frameworks **could not orchestrate multiple servers in parallel**, which is essential for large test campaigns.
  - **BIOS configuration was not automated**, forcing engineers to make changes manually and increasing the risk of misconfiguration.
  - There was **no integration with key CPU statistics and monitoring tools** (e.g., powerstat, turbostat, netstat, nohup), making it hard to correlate performance and system behavior.
- **Problem 4 – Complex and fragmented benchmark parameter space**
  - Each DPDK benchmark (for example, DPDK crypto) exposes **more than 10 performance-sensitive parameters**, with a mix of shared and workload-specific flags.
  - Users needed a **flexible framework** that supports both **standard "golden" configurations** and **custom experiments** without having to script everything from scratch.
- **Problem 5 – High domain and platform complexity**
  - The team initially had **limited DPDK domain knowledge**, and had to quickly ramp up on packet processing and DPDK internals.
  - The framework needed to support **multiple compilers** (gcc, AOCC, clang) and **multiple operating systems** (Ubuntu 22.04/24.04, RHEL 8/9), increasing complexity.

- **Overall opportunity – a unified AMD-centric benchmark framework**
  - There was a clear opportunity to build a **unified, AMD-centric benchmark automation framework** that:
    - Encodes **AMD-specific learnings, tuning, and best practices** for DPDK and networking workloads.
    - Automates **end-to-end setup, BIOS configuration, benchmark execution, statistics collection, and reporting** across platforms.
    - Enables **faster, more systematic, and more reliable performance analysis** of AMD CPUs across configurations and releases, supporting both internal optimization and external positioning.

---

2. My Developer Contribution – Role of Radheshyam

- **Lead role – end-to-end ownership (Radheshyam)**
  - **Radheshyam initiated and led** the automation effort for AMD's DPDK benchmarking framework.
  - Took **end-to-end ownership** from **domain research** to **framework design, implementation, integrations, and team guidance**.
  - Acted as the **primary technical point of contact** for questions on DPDK automation, BIOS automation, and benchmark workflows.

---

a) Domain research & AMD-centric documentation (Radheshyam)

- **Ramping up from zero DPDK background**
  - **Radheshyam started with zero DPDK experience** and quickly ramped up on:
    - DPDK packet processing concepts.
    - DPDK crypto workflows and other benchmarks (testpmd, L3/L2 forward, etc.).
  - Shared this knowledge with the team, helping others build confidence with DPDK workloads.
- **Adapting Intel-centric guides to AMD CPUs**
  - **Radheshyam analysed and reverse-engineered existing Intel-centric DPDK guides** and tuning recommendations.
  - Translated these into **AMD-specific guidance**, taking into account AMD CPU characteristics, BIOS options, and platform behaviors.
- **Creating AMD-centric internal documentation**
  - **Radheshyam authored internal AMD documentation** covering:
    - How to compile, configure, and run DPDK crypto and other benchmarks on AMD platforms.
    - **Best-known configurations** for various OS / BIOS / compiler combinations.
  - This documentation became a **reference for new team members** and reduced onboarding time for future DPDK work.

---

b) Framework enhancement & UI collaboration (Radheshyam)

- **Unifying the automation framework**
  - **Radheshyam enhanced the existing framework design** into a **single, unified automation framework** to handle:
    - Multiple DPDK benchmarks (crypto, core tests, testpmd, L3/L2 forward).
    - Additional network benchmarks such as **iperf2/iperf3**.
- **Parameter-driven UI collaboration**
  - **Radheshyam closely collaborated with the UI team** to make the framework **easily usable by non-experts**.
  - Helped design and validate a **parameter-driven UI** where users can:
    - Select the required benchmark and input benchmark-specific parameters.
    - Enable/disable different **CPU statistics tools** (powerstat, turbostat, netstat, nohup) via toggles.
    - Configure **BIOS options** such as SMT on/off, Turbo mode on/off, and C-states enabled/disabled.
  - Ensured that **sensible default configurations** were available for each benchmark to reduce user error.

---

c) Core development & automation – Ansible, BIOS, Xena (Radheshyam)

- **Reusable Ansible roles for DPDK benchmarks**
  - **Radheshyam developed reusable Ansible roles** for DPDK benchmarks that can be shared across:
    - DPDK crypto, core tests, testpmd, L3/L2 forward, and other workloads.
  - These roles standardized installation, configuration, and execution flows across benchmarks.
- **OS & package automation via Ansible (Radheshyam)**
  - **Radheshyam wrote Ansible modules and playbooks** to:
    - Install required OS packages, compilers (gcc, AOCC, clang), and statistics tools (netstat, powerstat, etc.).
    - Handle **multiple OS flavors** (Ubuntu 22.04/24.04, RHEL 8/9) using conditional tasks and variables.
  - This significantly reduced manual setup time and **ensured consistent environments** across servers.
- **BIOS automation for AMD platforms (Radheshyam)**
  - **Radheshyam implemented Python-based BIOS automation scripts**:
    - For **AMD Cinnabar platforms**, created scripts to set BIOS configurations programmatically.
    - For **Dell and HP servers**, implemented **Redfish API–based automation** to change BIOS settings remotely.
  - This allowed large test campaigns to **switch BIOS profiles automatically** without manual intervention.

- **Xena-based packet generator integration (Radheshyam)**
  - **Radheshyam designed and implemented a reusable Python module for Xena traffic generator integration**.
  - The module was written in a **plug-and-play style**, so it can be reused by any benchmark requiring packet generation.
  - **Radheshyam integrated this module with DPDK testpmd**, enabling automated end-to-end packet generation and measurement.
- **CLI framework for BIOS automation (Radheshyam)**
  - **Radheshyam built a Python-based BIOS automation CLI framework (using Clif)** to:
    - Provide a consistent command-line interface for manual vs automated BIOS operations.
    - Reduce errors during manual reconfiguration and speed up repetitive tasks.
  - This CLI became an **everyday tool for the team** when experimenting with different BIOS profiles.

---

d) Benchmark execution, stats collection & parsing (Radheshyam)

- **End-to-end automation of complex benchmarks**
  - **Radheshyam's** team fully automated all 7 networking benchmarks, and he personally designed and implemented 3 of them end-to-end (DPDK crypto, DPDK vhost, and DPDK testpmd), including high-complexity workloads integrated with the Xena hardware packet generator.
  - For example, a single DPDK crypto benchmark involved **23+ DPDK crypto commands**, each with **10+ variables**; **Radheshyam converted these into reusable templates** for repeatable execution.
- **Reusable stats processing modules (Radheshyam)**
  - **Radheshyam developed reusable Bash scripts and a Python module** to:
    - Capture CPU and system statistics in parallel using selected tools (e.g., `powerstat`, `turbostat`) via Bash scripts.
    - Process the raw statistics and store them in a structured database for downstream analysis using the Python module.
- **Custom parsers for DPDK benchmarks (Radheshyam)**
  - **Radheshyam wrote custom parsing scripts** for:
    - DPDK testpmd output.
    - DPDK crypto benchmark logs.
    - DPDK vhost benchmark data.
  - These parsers **normalized raw command outputs into structured metrics**, enabling reliable comparisons and dashboards.

---

e) Reporting, dashboards & comparison workflow (Radheshyam)

- **Data → dashboard integration (Radheshyam)**
  - **Radheshyam worked with the UI/dashboard team** to integrate processed benchmark data into a **database-backed reporting layer**.

    - Users can now:
        - View **graphs and performance metrics** for each run.
        - Drill down into key performance indicators for a specific benchmark and configuration.
- **Comparison flows across configurations (Radheshyam)**
    - **Radheshyam helped design the comparison workflow**, so users can:
        - Select and compare **multiple runs side by side** (e.g., different BIOS configs, OS versions, compilers, CPU SKUs).
        - Quickly understand **performance impact** of changes like SMT on vs off, different BIOS versions, or OS upgrades.
- **Tracking performance across releases (Radheshyam)**
    - **Radheshyam continuously tracked CPU performance trends** across OS, BIOS, and hardware updates.
    - Helped the wider team understand **how configuration changes affected AMD CPU performance** over time.

---

f) Team leadership, mentoring & unblocker role (Radheshyam)

- **Technical leadership in a 3-member team (Radheshyam)**
    - **Radheshyam acted as the technical lead** for a 3-member development team.
    - Assigned exploration tasks, reviewed designs, and guided investigations into new benchmarks and tools.
- **Unblocking the team on complex issues**
    - **Radheshyam regularly unblocked teammates** facing issues with:
        - DPDK configuration and debugging.
        - OS variations (Ubuntu vs RHEL).
        - BIOS configuration and Redfish integrations.
    - This ensured **steady progress and reduced delays** in the overall project.
- **Driving usability and framework flexibility (Radheshyam)**
    - **Radheshyam proposed improvements** to user experience, configuration flexibility, and reusability within the framework.
    - Incorporated feedback from users and stakeholders to make the framework **easier to operate and extend** for future benchmarks.

---

3. Your Impact

Awesome, this is the **Impact** part – I'll keep it in bullets with small headings and weave in **Radheshyam's name** wherever it fits naturally.

---

3. Impact of Radheshyam's Contribution

- **Reliable AMD-centric automation framework (Radheshyam's ownership)**
  - As a result of **Radheshyam's work**, the team gained a **reliable, AMD-centric, end-to-end automation framework** to run DPDK and networking benchmarks across multiple platforms, OS versions, BIOS versions, and compilers.
  - The framework is now the **default way** for the team to execute AMD DPDK/Networking benchmark campaigns.
- **Manual work replaced by a single automated pipeline (driven by Radheshyam)**
  - **Radheshyam's automation** replaced multiple manual, error-prone steps with a **single, repeatable pipeline**, including:
    - BIOS changes and profile switching.
    - Dependency and tool installation.
    - Running individual benchmarks with many parameters.
    - Collecting and processing CPU and system statistics.
  - This significantly **reduced human error and setup time** for each campaign.
- **Scalable, parameter-driven experimentation (enabled by Radheshyam)**
  - Thanks to the framework designed and implemented by **Radheshyam**, engineers can now:
    - Define **10–50+ test scenarios in one go**, run them automatically, and receive consolidated results at the end.
    - Execute **multi-server benchmarking scenarios in parallel**, which **was not possible** with the earlier automation framework.
  - This made large-scale experimentation **practical and repeatable**, instead of ad-hoc.
- **Faster performance tuning and insight generation (led by Radheshyam)**
  - **Radheshyam's work** made it **much faster to explore performance tuning** across:
    - BIOS features (SMT on/off, turbo, C-states).
    - OS choices (Ubuntu vs RHEL).
    - Compiler options (gcc, AOCC, clang).
  - The team can now more easily **see how each configuration impacts AMD CPU performance**, leading to **data-driven tuning decisions**.
- **Reusable knowledge and assets for future teams (authored by Radheshyam)**
  - The documentation, Ansible roles, BIOS automation scripts, parsing modules, and integration patterns created by **Radheshyam** are now:
    - Used to **onboard new team members** into DPDK and AMD benchmarking.
    - Reused to **extend the framework to new benchmarks and future AMD platforms** with minimal extra effort.
- **Overall transformation (driven by Radheshyam)**
  - Overall, **Radheshyam's contribution transformed a manual, fragmented benchmarking process into a structured, automated, and observable system**, enabling **AMD-focused performance characterization at scale** and improving how the team runs and reasons about networking benchmarks on AMD CPUs.

Nice, this fits perfectly as a reflection section. Here's your **Lessons Learned** rewritten with headings, clean language, and **Radheshyam** mentioned clearly.

---

4. Lessons Learned – Reflections from Radheshyam

- **Start simple, then generalize (Radheshyam's approach)**
  - At the beginning, **Radheshyam deliberately focused on a single benchmark (DPDK crypto)** instead of trying to automate everything at once.
  - As **Radheshyam** added more DPDK benchmarks and OS/BIOS variations, he saw **how even small configuration changes could significantly impact performance**.
  - This taught **Radheshyam** to validate patterns on one benchmark first, then carefully generalize them across workloads and platforms.
- **Deep domain understanding is essential for good automation (lesson for Radheshyam)**
  - **Radheshyam started with almost no DPDK knowledge**, and initially it was tempting to just "script commands".
  - The project reinforced for **Radheshyam** that **effective automation requires real domain understanding**, not just tooling skills.
  - By investing time in DPDK internals and **AMD-specific tuning**, **Radheshyam** significantly improved both:
    - The **quality and usefulness of the framework**, and
    - **His own technical depth** in networking and performance engineering.
- **Infrastructure and BIOS automation are first-class citizens (realization by Radheshyam)**
  - Initially, BIOS and server setup were treated as **manual prerequisites** outside the "real" automation.
  - Through this project, **Radheshyam realized** that **BIOS and infrastructure automation are critical** for reliability and repeatability.
  - **Radheshyam learned and implemented Ansible for the first time**, built CLI-based automation that actually went to production, and saw how this removed entire classes of manual errors.
- **Team collaboration and knowledge sharing accelerate maturity (growth for Radheshyam)**
  - Since there were no strong DPDK domain experts in the team initially, **Radheshyam learned to drive collaborative learning**, including:
    - Breaking down research tasks and assigning them to team members.
    - Sharing findings via **documentation, walkthroughs, and discussions**.
    - Continuously updating the framework and docs as **new behaviours and corner cases were discovered**.
  - This experience taught **Radheshyam** how **collaboration, not solo work, accelerates maturity of both the solution and the team**.

---

5. Skills Demonstrated (for mapping)

You can map these to **Core / Common / Specialist** skills in the IBM badge form.

Core skills (Experienced level):

- **Problem solving & structured thinking** – Took an unstructured problem (no AMD DPDK docs, fragmented tools) and defined a clear automation architecture and execution plan.
- **Disciplined development practices** – Used scripts, Ansible, and standard patterns to ensure repeatable, testable automation instead of ad-hoc manual steps.
- **Technical leadership** – Led a 3-member team, guided research, unblocked others, and proposed improvements to framework design and user experience.

Common skills (Foundation level):

- **Collaboration & communication** – Worked with teammates to share DPDK knowledge, document AMD-centric procedures, and define UX for the framework and dashboards.
- **Growth mindset & continuous learning** – Ramp-up from zero DPDK/domain knowledge to building a robust framework and documentation through constant learning and experimentation.
- **Documentation & knowledge sharing** – Created AMD-centric DPDK run books and internal guides that others can use and extend.

Specialist skills (as applicable):

- **High-performance networking & DPDK** – Worked with DPDK crypto and multiple DPDK benchmarks for AMD CPU performance characterization.
- **Automation & DevOps tooling** – Implemented Ansible-based automation for OS, compilers, benchmarks, and stats tools; integrated Redfish for BIOS configuration.
- **Linux systems & multi-OS support** – Handled automation across Ubuntu 22.04/24.04 and RHEL 8/9, adapting to OS-specific package and configuration differences.
- **Performance testing & observability** – Designed parameter-driven experiments, integrated stats tools (powerstat, turbostat, netstat), and built flows to turn raw data into dashboards and comparisons.